DETAILED PROBLEM STATEMENT

Smart Traffic Signal Scheduling System Using Stack and Queue

1. Introduction

Traffic intersections need a clear and orderly method for giving different roads their turn to move. A scheduling model can help demonstrate how traffic signals are processed in sequence. In this DSA mini project, a queue is used to manage the order in which signals receive a turn, while a stack records the history of activated signals so the most recently activated signal can be reviewed first.

2. Problem Definition

Design and implement a Smart Traffic Signal Scheduling System in C++ using both a queue and a stack. Each signal stores a road/direction name and a green-light duration. The queue follows First In, First Out (FIFO), so the signal at the front is served first. After its simulated turn, that signal is added to the rear of the queue so that the scheduling cycle can continue. At the same time, each signal that completes its turn is pushed onto a stack. The stack follows Last In, First Out (LIFO), allowing the program to display the most recently activated signals first.

The program should accept signal details from the user, display the initial queue, simulate signal activation one at a time, allow the user to stop the schedule, and optionally display the signal history stored in the stack.

3. Need for the System

A queue is appropriate for managing a normal sequence because each signal waits for its turn in arrival order. A stack provides a simple history mechanism that displays the latest completed signal first. Using both structures demonstrates two fundamental data-structure concepts in one practical simulation and separates scheduling from history tracking.

4. Objectives

- Represent each road signal with its name and green-light duration.
- Use a queue to schedule signals in FIFO order.
- Reinsert a completed signal at the rear of the queue to repeat the cycle.
- Use a stack to store completed signal activations in LIFO order.
- Display the queue and active signal clearly.
- Allow the user to stop the scheduling simulation.
- Display signal history with the most recent activation first.
- Demonstrate queue and stack operations in a C++ DSA application.

5. Proposed Working

First, the program reads the number of signals and each signal's name and positive green duration. Every signal is inserted into the queue in the order entered. During scheduling, the program removes the front signal and displays it as GREEN. When the user presses Enter to complete the simulated interval, the signal is pushed onto the history stack and inserted at the rear of the queue. This creates a repeating FIFO schedule. If the user enters q, scheduling stops and the currently selected signal is restored to the front of the queue. The history can then be displayed by repeatedly reading a temporary copy of the stack from top to bottom.

6. Data Structures Used

Queue: Stores signals waiting for their turn. The front signal is processed first, and completed signals are reinserted at the rear. **Stack:** Stores the signals that have completed a simulated turn. Its top contains the most recently activated signal. The program uses C++ Standard Library containers std::queue and std::stack.

7. Functional Requirements

- Accept a positive number of signals from the user.
- Accept a non-empty name and positive green duration for each signal.
- Display the queue from front to rear.
- Activate the front signal and display its configured duration.
- After a completed turn, push the signal to the stack and requeue it.
- Stop scheduling when the user enters q.
- Display the completed-signal history in LIFO order.

8. Scope and Limitations

This is an educational software simulation. It does not operate real traffic lights, measure live vehicle density, or provide certified traffic-control decisions. In this interactive version, pressing Enter advances to the next signal; the duration is displayed but is not automatically timed. The queue order is the order in which the user enters the signals, and the stack is used only as a history record.

9. Expected Outcome

The completed program should maintain a queue of traffic signals, repeatedly serve signals in FIFO order, record completed activations in a stack, and display history in reverse completion order. It should validate basic input and terminate cleanly when requested.